شوگاریستا · مستندات فنی

راهنمای استقرار ایمیج‌محور

از لپ‌تاپ تا سرور — بدون git، بدون build روی سرور، بدون خطر از دست رفتن دیتا

نسخه ۱.۰ ۱۶ آبان ۱۴۰۵ مخاطب: مدیر سیستم / توسعه‌دهنده

فهرست

1. تصویر کلی
2. فایل‌هایی که اضافه شدند
3. انتخاب رجیستری و لاگین
4. تنظیم `.env`
5. اولین Release
6. آماده‌سازی سرور
7. چرخهٔ روزمره
8. برگشت به نسخهٔ قبلی
9. بکاپ خودکار
10. نکات و تله‌ها
11. چک‌لیست هر دیپلوی
12. چرا این روش امن است

۰ تصویر کلی

در این روش، ایمیج‌ها روی لپ‌تاپ ساخته و به یک رجیستری (مثل GHCR) فرستاده می‌شوند. سرور فقط آن‌ها را **pull** می‌کند و اجرا می‌کند. هیچ سورس‌کدی روی سرور نیست، هیچ buildی انجام نمی‌شود و هیچ gitی در کار نیست.



نکتهٔ کلیدی

دیتابیس و فایل‌های آپلودی در **named volume**هستند. با هر دیپلوی، کانتینرها عوض می‌شوند ولی volumeها دست‌نخورده می‌مانند. تنها دستوری که می‌تواند دیتا را پاک کند `docker compose down -v` است — هرگز آن را اجرا نکنید.

۱ فایل‌هایی که اضافه شدند

فایل	کجا اجرا می‌شود	کار
`docker-compose.prod.yml`	سرور	استک فقط-ایمیج: بدون `build`، بدون bind mount
`deploy/nginx/Dockerfile`	لپ‌تاپ	کانفیگ nginx را داخل ایمیج می‌پزد
`scripts/release.sh`	لپ‌تاپ	۴ ایمیج را برای `linux/amd64` می‌سازد و push می‌کند
`scripts/server-deploy.sh`	سرور	pull → بکاپ → up → health → smoke test
`scripts/backup-db.sh`	هر دو	بکاپ فشردهٔ دیتابیس با نگه‌داری N نسخهٔ آخر
`scripts/restore-db.sh`	هر دو	بازگردانی کامل دیتابیس از بکاپ

تفاوت‌های مهم با فایل توسعه

- همهٔ سرویس‌ها `image:` دارند و هیچ‌کدام `build:` ندارند.

- `public-ui` دیگر پوشهٔ `Sugarista-Theme` را mount نمی‌کند؛ فایل‌ها داخل ایمیج هستند.
- `proxy` دیگر `deploy/nginx/default.conf` را mount نمی‌کند؛ کانفیگ داخل ایمیج است.
- `name: sigarista` قفل شده تا نام volume هرگز عوض نشود.

۲ انتخاب رجیستری و لاگین

پیشنهاد: **GitHub Container Registry (GHCR)** — چون ریپو همان‌جاست و رایگان است.

۲-۱) ساخت توکن

GitHub → **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)** → Generate new token

- برای لپ‌تاپ (push): دسترسی `write:packages`
- برای سرور (pull): دسترسی `read:packages`

۲-۲) لاگین روی لپ‌تاپ

```
echo "TOKEN" | docker login ghcr.io -u aref-zangeneh --password-stdin
```

۲-۳) گزینهٔ جایگزین: Docker Hub

```
docker login docker.io -u <username>
# و در .env:  IMAGE_REGISTRY=docker.io/<username>
```

۳ تنظیم `.env` روی لپ‌تاپ

فقط این خط لازم است (پیش‌فرض هم همین است):

```
IMAGE_REGISTRY=ghcr.io/aref-zangeneh
```

۴ اولین Release روی لپ‌تاپ

```
cd ~/Desktop/Sugarista-Project
./scripts/release.sh
```

خروجی مورد انتظار:

```
==> Building sugarista-api (linux/amd64)
==> Building sugarista-admin (linux/amd64)
==> Building sugarista-theme (linux/amd64)
==> Building sugarista-proxy (linux/amd64)
==> Pushed all images with tag 2026.11.07-1430

Registry : ghcr.io/aref-zangeneh
Tag      : 2026.11.07-1430  (also tagged :latest)
Platform : linux/amd64
```

زمان ساخت

بار اول برای ایمیج API روی مک حدود ۴ تا ۵ دقیقه طول می‌کشد (شبیه‌سازی معماری amd64). بارهای بعد به‌خاطر کش بسیار سریع‌تر است.

گزینه‌های مفید

```
IMAGE_TAG=v1.2 ./scripts/release.sh                # تگ دلخواه
PUSH=0 ./scripts/release.sh                        # فقط build، بدون push
TARGET_PLATFORM=linux/arm64 ./scripts/release.sh   # اگر سرور ARM بود
```

معماری

سرور x86_64 است، پس همیشه `TARGET_PLATFORM=linux/amd64` (پیش‌فرض) درست است. ایمیج arm64 روی سرور اجرا نمی‌شود.

۵ آماده‌سازی سرور (فقط یک‌بار)

۵-۱) نام volume فعلی را بررسی کنید

این مرحله حیاتی است؛ اگر نام عوض شود، استک جدید یک دیتابیس **خالی** می‌سازد.

```
docker volume ls | grep postgres-data
```

باید چیزی مثل `sigarista_postgres-data` ببینید؛ یعنی پیشوند پروژه `sigarista` است که با `name: sigarista` در فایل جدید هم‌خوان است. اگر پیشوند چیز دیگری بود، در `.env` سرور اضافه کنید:

```
COMPOSE_PROJECT_NAME=<همان-پیشوند>
```

۵-۲) یک بکاپ اضافه بگیرید

```
cd /opt/sugarista/Sigarista
docker compose exec -T postgres pg_dump -U sugarista -d sugarista \
  --no-owner --no-privileges | gzip > /root/pre-image-deploy.sql.gz
ls -lh /root/pre-image-deploy.sql.gz
```

۵-۳) فایل‌ها را از لپ‌تاپ کپی کنید

```
# روی لپتاپ
cd ~/Desktop/Sugarista-Project
scp docker-compose.prod.yml root@sugarista-web:/opt/sugarista/Sigarista/
scp -r scripts root@sugarista-web:/opt/sugarista/Sigarista/
```

۵-۴) لاگین رجیستری روی سرور

فقط اگر ایمیج‌ها خصوصی هستند:

```
echo "TOKEN_READ_PACKAGES" | docker login ghcr.io -u aref-zangeneh --password-stdin
```

این کار یک‌بار انجام می‌شود و در `~/.docker/config.json` می‌ماند.

۵-۵) استک قدیمی را خاموش کنید — بدون `-v`

```
cd /opt/sugarista/Sigarista
docker compose down
```

۵-۶) اولین دیپلوی ایمیجی

```
cd /opt/sugarista/Sigarista
chmod +x scripts/*.sh
./scripts/server-deploy.sh
```

خروجی موفق:

```
==> Target image tag: latest
==> Backing up database
==> Pulling images
==> Applying changes
==> Waiting for API health
==> Smoke test
  /health  -> 200
  /admin/  -> 200
  /        -> 200
==> Deployed image tag latest successfully.
```

از این لحظه، سرور فقط ایمیج اجرا می‌کند. سورس‌کد قدیمی را می‌توانید پاک کنید — به‌جز این‌ها:

```
/opt/sugarista/Sigarista/
├── .env                    ← رمزها، دست نزنید
├── docker-compose.prod.yml
├── scripts/
└── backups/
```

۶ چرخهٔ روزمره

روی لپ‌تاپ — بعد از هر تغییر کد

```
cd ~/Desktop/Sugarista-Project
./scripts/release.sh
```

روی سرور

```
cd /opt/sugarista/Sigarista
./scripts/server-deploy.sh
```

تمام. بدون git، بدون pull، بدون build روی سرور.

۷ برگشت به نسخهٔ قبلی (Rollback)

هر release یک تگ نسخه‌دار هم push می‌کند، پس برگشت ساده است:

```
cd /opt/sugarista/Sigarista
IMAGE_TAG=2026.11.06-0900 ./scripts/server-deploy.sh
```

متغیر محیطی بر `.env` اولویت دارد، پس لازم نیست فایل را دست بزنید. برای دیدن نسخه‌های موجود: GitHub → هر پکیج → **Packages** → **Versions**.

> **اگر مایگریشن اجرا شده بود**
>
> اول کد را برگردانید. اگر مشکل ادامه داشت، دیتابیس را از بکاپ بازگردانید:
>
> `./scripts/restore-db.sh backups/sugarista-2026xxxx-xxxxxx.sql.gz`

۸ بکاپ خودکار روزانه

```
crontab -e
```

```
0 3 * * * cd /opt/sugarista/Sigarista && COMPOSE_FILE=/opt/sugarista/Sigarista/docker-compose.prod.yml ./scripts/backup-db.sh >> /var/log/sugarista-backup.log 2>&1
```

و کپی offsite:

```
0 4 * * * rclone copy /opt/sugarista/Sigarista/backups/ remote:sugarista-backups/ --max-age 48h
```

> **قانون طلایی بکاپ**
>
> بکاپی که یک‌بار restore نشده باشد، بکاپ نیست. ماهی یک‌بار روی یک محیط تست، restore را تمرین کنید.

۹ نکات و تله‌ها

موضوع	نکته
معماری	سرور x86_64 است؛ همیشه از `linux/amd64` استفاده کنید
نام پروژه	`name: sigarista` قفل شده؛ پوشه را جابه‌جا نکنید.
`down -v`	هرگز. تنها دستوری است که دیتا را می‌کشد.
تغییر تم	چون bind mount حذف شد، نیاز به release جدید دارد.
تغییر nginx	کانفیگ داخل ایمیج proxy پخته می‌شود؛ بعد از تغییر، release بزنید.
`.env`	هرگز داخل ایمیج نمی‌رود و با دیپلوی عوض نمی‌شود.
مایگریشن	API روی استارت خودش `MigrateAsync()` را اجرا می‌کند.
رجیستری خصوصی	بدون `docker login` روی سرور، pull با `unauthorized` شکست می‌خورد.
فضای دیسک	هر release یک ایمیج جدید می‌آورد؛ گاهی `docker image prune -f` بزنید
رمز ادمین	با اولین seed ست می‌شود و با دیپلوی عوض نمی‌شود.

۱۰ چک‌لیست هر دیپلوی

لپ‌تاپ

- ☐ `dotnet test` سبز است
- ☐ `npm run build` در `Sugarista-Admin` بدون خطاست
- ☐ `./scripts/release.sh` بدون خطا تمام شد و تگ نسخه را یادداشت کردم

سرور

- ☐ `df -h` حداقل ۲ تا ۳ گیگ فضای آزاد نشان می‌دهد
- ☐ `./scripts/server-deploy.sh` هر سه endpoint را ۲۰۰ داد
- ☐ یک‌بار `/admin/` را باز کردم و لاگین گرفتم

۱۱ چرا این روش امن است

- **دیتابیس و آپلودها** در volume هستند و اصلاً به کد کاری ندارند.

- **سرور هیچ‌وقت کامپایل نمی‌کند** → نه NuGet قطع می‌شود، نه npm، نه رم سرور.
- **هر نسخه یک ایمیج تغییرناپذیر** است → rollback دقیقاً همان چیزی را برمی‌گرداند که قبلاً کار می‌کرد.
- `.env` **و رمزها** هرگز وارد ایمیج یا ریپو نمی‌شوند.
- **بکاپ خودکار** قبل از هر دیپلوی گرفته می‌شود.

برگهٔ مرجع سریع

می‌خواهم...	دستور
نسخهٔ جدید بسازم	`./scripts/release.sh`
روی سرور مستقر کنم	`./scripts/server-deploy.sh`
به نسخهٔ قبل برگردم	`IMAGE_TAG=<old> ./scripts/server-deploy.sh`
بکاپ بگیرم	`./scripts/backup-db.sh`
بکاپ را برگردانم	`./scripts/restore-db.sh backups/<file>.sql.gz`
لاگ API را ببینم	`docker compose -f docker-compose.prod.yml logs -f api`
وضعیت را ببینم	`docker compose -f docker-compose.prod.yml ps`
